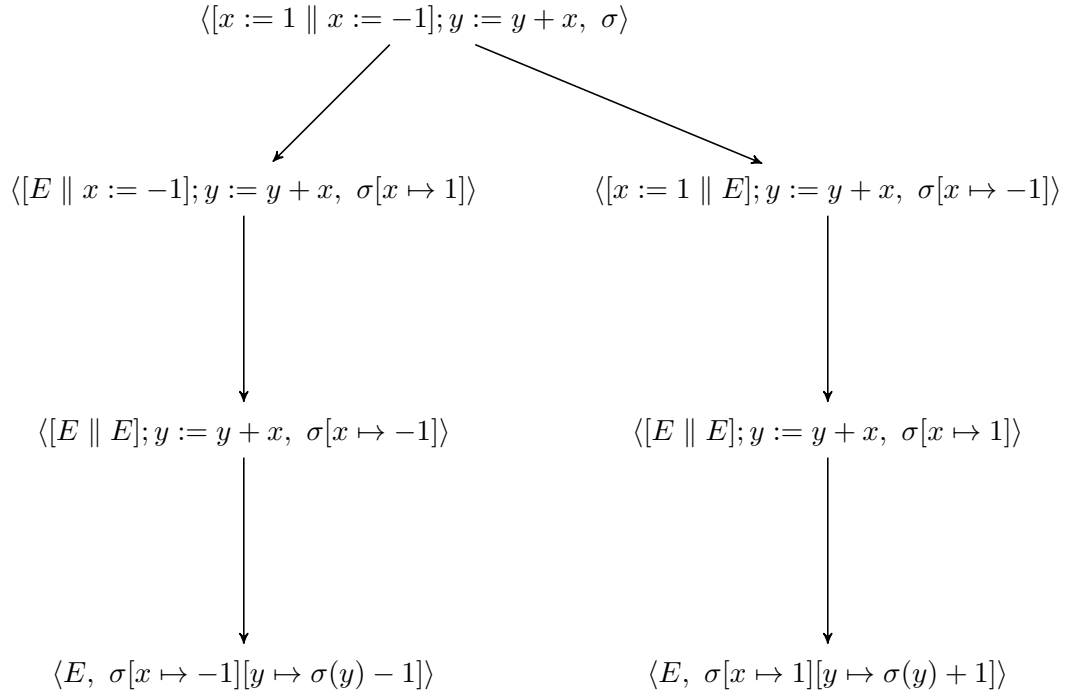


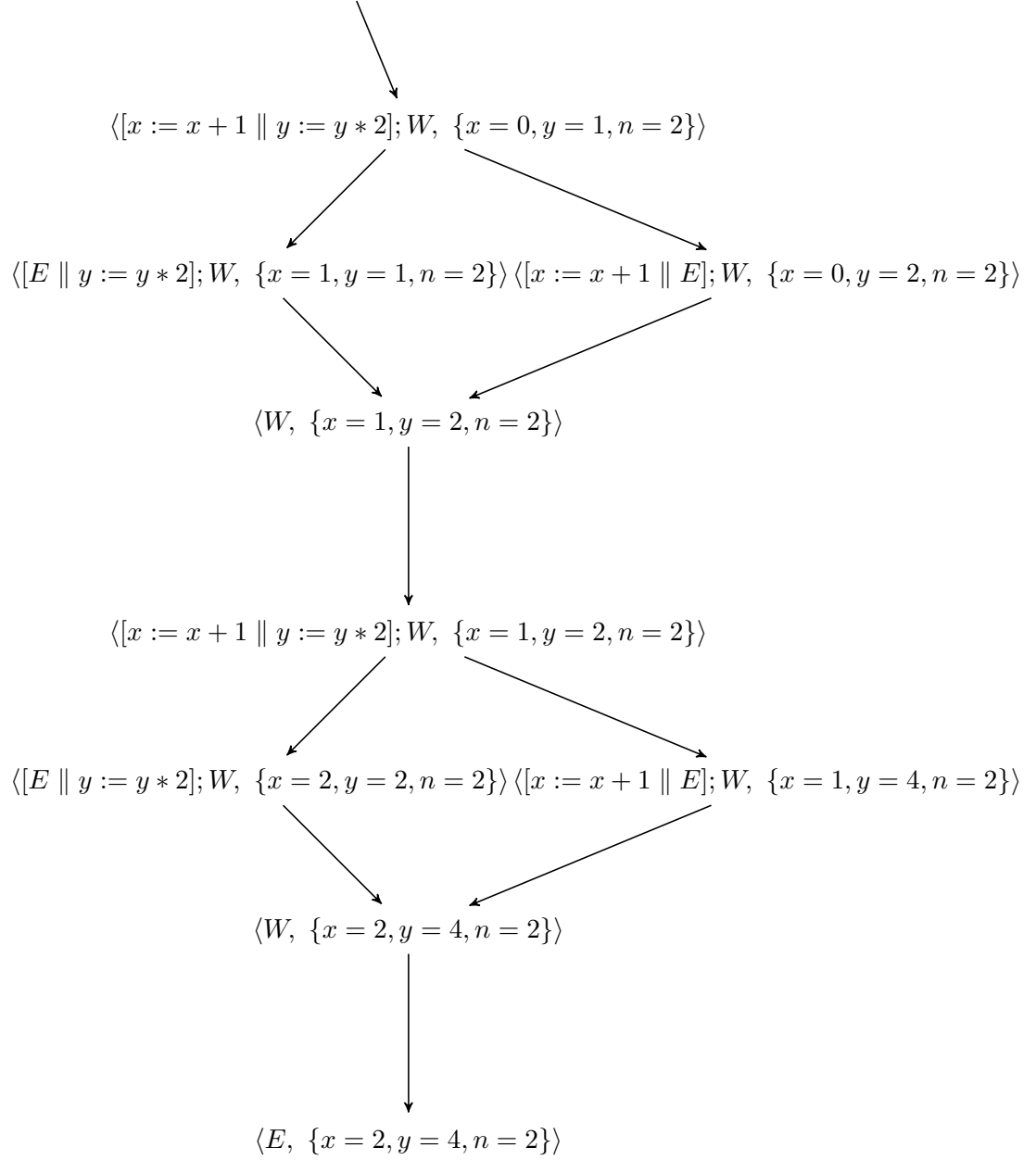
CS536 Science of Programming
Fall 2025
Sample Solution Sketches for the Practice of Parallel Programs

1. Here is an evaluation graph for configuration $\langle [x := 1 \parallel x := -1]; y := y + x, \sigma \rangle$.



2. Here is an evaluation graph for configuration $\langle W, \{x = 0, y = 1, n = 2\} \rangle$ where $W \equiv \mathbf{while} \ x < n \ \mathbf{do} \ [x := x + 1 \parallel y := y * 2] \ \mathbf{od}$.

$\langle \mathbf{while} \ x < n \ \mathbf{do} \ [x := x + 1 \parallel y := y * 2] \ \mathbf{od}, \ \{x = 0, y = 1, n = 2\} \rangle$



3. We can create the following tests for disjointedness and disjoint conditions.

i	j	$change(S_i)$	$vars(S_j)$	$free(p_j, q_j)$	$S_i \text{ int } S_j ?$	$S_i \text{ int cond}_j ?$
1	2	y	z	x, z	<i>No</i>	<i>No</i>
2	1	z	x, y	x, y, z	<i>No</i>	<i>Yes</i>

From the test results, we can see that:

- (a) These two threads are disjoint.
- (b) These two threads don't have disjoint conditions since S_2 interferes with conditions of S_1 .
- (c) To use the Parallel Rule while proving a disjoint parallel program without disjoint conditions, we can age all the free variables in the preconditions of all threads, and we can get the following full proof outline.

$$\begin{aligned}
& \{z_0 = x_0 = 0 \wedge x_0 \leq 0\} \\
& [\{z_0 = x_0 = 0\} y := x_0 + 2 \{z_0 = x_0 = 0 \wedge y = x_0 + 2\} \parallel \\
& \quad \{x_0 \leq 0\} z := 1 \{z > x_0\}] \\
& \{z_0 = x_0 = 0 \wedge y = x_0 + 2 \wedge z > x_0\}
\end{aligned}$$

There are no threads in the above program write x , but we still aged it in the precondition; so whenever x is used in the program we replace x by x_0 (unless it is understood that x_0 is the aged version of x). Another way to write this proof outline correctly is to add $x = x_0$ in the precondition for those threads that read x .

$$\begin{aligned}
& \{z_0 = x_0 = 0 \wedge x = x_0 \wedge x_0 \leq 0\} \\
& [\{z_0 = x_0 = 0 \wedge x = x_0\} y := x + 2 \{z_0 = x_0 = 0 \wedge x = x_0 \wedge y = x_0 + 2\} \parallel \\
& \quad \{x_0 \leq 0\} z := 1 \{z > x_0\}] \\
& \{z_0 = x_0 = 0 \wedge x = x_0 \wedge y = x_0 + 2 \wedge z > x_0\}
\end{aligned}$$

4. First, let us show that the program is a disjoint parallel program.

i	j	$change(S_i)$	$vars(S_j)$	$S_i \text{ int } S_j ?$
1	2	y	z	<i>No</i>
2	1	z	x, y	<i>No</i>

Then, we can give the following formal proof.

1. $\{y = 2 \wedge z \geq 0\} z := 0 \{y = 2 \wedge z \geq x\}$ backward assignment
2. $\{x + 2 = 2 \wedge z \geq 0\} y := x + 2 \{y = 2 \wedge z \geq 0\}$ backward assignment
3. $\{x + 2 = 2 \wedge z \geq 0\} y := x + 2; z := 0 \{y = 2 \wedge z \geq x\}$ sequence 2,1
4. $z = x = 0 \Rightarrow x + 2 = 2 \wedge z \geq 0$ predicate logic
5. $\{z = x = 0\} y := x + 2; z := 0 \{y = 2 \wedge z \geq x\}$ strengthen precondition 4,3
6. $\{z = x = 0\} [y := x + 2 \parallel z := 0] \{y = 2 \wedge z \geq x\}$ sequentialization 5

Line 4 in the above proof is obvious.

5. (a) To test whether S_1^* interferes with S_2^* , we need to test to validity of the following triples:

- $\{p_2 \wedge q_1\} < T_1 > \{q_1\}$
- $\{p_2 \wedge q_3\} < T_1 > \{q_3\}$
- $\{p_2 \wedge q_4\} < T_1 > \{q_4\}$
- $\{p_3 \wedge q_1\} \text{ skip } \{q_1\}$
- $\{p_3 \wedge q_3\} \text{ skip } \{q_3\}$
- $\{p_3 \wedge q_4\} \text{ skip } \{q_4\}$

As an aside, the last three triples are trivially valid.

- (b) To test whether S_2^* interferes with S_1^* , we need to test to validity of the following triples:

- $\{q_1 \wedge p_1\} < T_2 > \{p_1\}$
- $\{q_1 \wedge p_2\} < T_2 > \{p_2\}$
- $\{q_1 \wedge p_3\} < T_2 > \{p_3\}$
- $\{q_1 \wedge p_4\} < T_2 > \{p_4\}$
- $\{q_3 \wedge p_1\} < T_3 > \{p_1\}$
- $\{q_3 \wedge p_2\} < T_3 > \{p_2\}$
- $\{q_3 \wedge p_3\} < T_3 > \{p_3\}$
- $\{q_3 \wedge p_4\} < T_3 > \{p_4\}$

6. First, let us test whether S_1^* and S_2^* are interference-free.

- $\{x \geq 0 \wedge z \geq 3\} < x := x + 1; y := x + 2 > \{z \geq 3\}$ is totally valid.
- $\{x \geq 0 \wedge x \geq 3\} < x := x + 1; y := x + 2 > \{x \geq 3\}$ is totally valid.
- $\{x \geq 0 \wedge y \geq 3\} < x := x + 1; y := x + 2 > \{y \geq 3\}$ is totally valid.
- $\{z \geq 3 \wedge x \geq 0\} x := z \{x \geq 0\}$ is totally valid.
- $\{z \geq 3 \wedge y \geq 3\} x := z \{y \geq 3\}$ is totally valid.
- $\{x \geq 3 \wedge x \geq 0\} y := x \{x \geq 0\}$ is totally valid.
- $\{x \geq 3 \wedge y \geq 3\} y := x \{y \geq 3\}$ is totally valid.

We can see that S_1^* and S_2^* are interference free, so we can apply the parallelism rule to their parallel composition.

Thus, we can give the following full proof outline. I added a postcondition in S_1^* calculated from the forward assignment rule, so that it become full proof outlines. S_2^* is already a full proof outline.

$$\begin{aligned}
& \{x \geq 0 \wedge z \geq 0\} \\
& [\{x \geq 0\} < x := x + 1; y := x + 2 > \{x_0 \geq 0 \wedge x_1 = x_0 + 1 \wedge y = x_1 + 2\} \{y \geq 3\} \parallel \\
& \quad \{z \geq 3\} z := x \{x \geq 3\} y := x \{y \geq 3\}] \\
& \{y \geq 3\}
\end{aligned}$$

7. We can give the following formal proof.

1. $\{k > 0 \wedge x = 1\} y := 0 \{k > 0 \wedge x = 1 \wedge y = 0\}$ forward assignment
2. $k > 0 \wedge x = 1 \wedge y = 0 \Rightarrow y < k$ predicate logic
3. $\{k > 0 \wedge x = 1\} y := 0 \{y < k\}$ weaken postcondition 1,2
4. $\{k > 0\} \text{ await } x = 1 \text{ then } y := 0 \text{ end } \{y < k\}$ await 3

8. These are the possible deadlock conditions for this parallel program.

- $y = k + 1 \wedge x = 1 \wedge y \neq 0 \wedge y = 0 \wedge y < k$ (this is $p_1 \wedge \neg B_1 \wedge q_2$), which is unsatisfiable.
- $k > 0 \wedge x \neq 1 \wedge x = 1$ (this is $p_2 \wedge \neg B_2 \wedge q_1$), which is unsatisfiable.
- $y = k + 1 \wedge x = 1 \wedge y \neq 0 \wedge k > 0 \wedge x \neq 1$ (this is $p_1 \wedge \neg B_1 \wedge p_2 \wedge \neg B_2$), which is unsatisfiable.

Since all the possible deadlock conditions are unsatisfiable, this parallel program is deadlock-free.